```
separate (IDL.SEM_PHASE)¶    --|-----------------------------------------------------------------------------------------¶    --| REQ_UTIL¶    --
|--------------------------------------------------------------------------------------------¶package body REQ_UTIL is¶  use SET_UTIL;¶  use DEF_UTI
L;¶¶    --|------------------------------------------------------------------------------------¶    --| REQ_GENE¶    --|--------------------
--------------------------------------------------------------------¶  package¬body REQ_GENE is¶¶    --||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||¶    --|¶   procedure REQ_DEF_XXX (EXP : TREE; DEFSET : in out DEFSET_TYPE) is¶¬     -- REMOV
E FROM DEFSET THOSE INTERPRETATIONS¬FOR WHICH¬IS_XXX FALSE¶¶     function REQUIRE_XXX (DEFSET : DEFSET_TYPE)¬return DEFSET_TYPE is¶      SET_TAIL :
 DEFSET_TYPE;¶       SET_HEAD : DEFINTERP_TYPE;¶       NEW_TAIL : DEFSET_TYPE;¶     begin¶       SET_TAIL :=¬DEFSET;¶       POP (SET_TAIL, SET_HE
AD);¶       if IS_EMPTY¬(SET_TAIL) then¶¬NEW_TAIL := SET_TAIL;¶       else¶¬NEW_TAIL := REQUIRE_XXX (SET_TAIL);¶       end if;¶       if IS_XXX (G
ET_DEF (SET_HEAD)) then¶¬if NEW_TAIL = SET_TAIL then¶¬  return DEFSET;¶¬else¶¬  NEW_TAIL := INSERT (NEW_TAIL, SET_HEAD);¶¬  return NEW_TAIL;¶¬end if;¶
       else¶¬return NEW_TAIL;¶       end if;¶¶     end¬REQUIRE_XXX;¶¶   begin¶     if IS_EMPTY (DEFSET) then¶       return;¶     end¬if;¶
DEFSET := REQUIRE_XXX (DEFSET);¶     if IS_EMPTY (DEFSET) then¶       ERROR (D (LX_SRCPOS, EXP), MESSAGE);¶     end¬if;¶   end REQ_DEF_XXX;¶
--||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶    --|¶   procedure REQ_TYPE_XXX (EXP : TREE;¬TYPES
ET :¬in out TYPESET_TYPE) is¬    --| ENLVE DE TYPESET LES INTERPRETATIONS QUI ONT IS_XXX FAUSSE¶¶     function REQUIRE_XXX (TYPESET : TYPESET_TYPE)
return TYPESET_TYPE is¶      SET_TAIL : TYPESET_TYPE;¶       SET_HEAD : TYPEINTERP_TYPE;¶       NEW_TAIL : TYPESET_TYPE;¶     begin¶       SET_T
AIL :=¬TYPESET;¶       POP (SET_TAIL, SET_HEAD);¶       if IS_EMPTY¬(SET_TAIL) then¶¬NEW_TAIL := SET_TAIL;¶       else¶¬NEW_TAIL := REQUIRE_XXX (SE
T_TAIL);¶       end if;¶       if IS_XXX (GET_TYPE (SET_HEAD))¬then¶¬if NEW_TAIL = SET_TAIL then¶¬  return TYPESET;¶¬else¶¬  NEW_TAIL := INSERT (NEW
_TAIL, SET_HEAD);¶¬  return NEW_TAIL;¶¬end if;¶       else¶¬return NEW_TAIL;¶       end if;¶¶     end¬REQUIRE_XXX;¶¶   begin¶     if IS_EMPTY (TY
PESET) then¶       return;¶     end¬if;¶     TYPESET := REQUIRE_XXX (TYPESET);¶     if IS_EMPTY (TYPESET) then¶       ERROR (D (LX_SRCPOS, EXP),
MESSAGE);¶     end¬if;¶   end REQ_TYPE_XXX;¶¶  end REQ_GENE;¶¶    --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||¶    --|¶  function GET_BASE_STRUCT (TYPE_SPEC :¬TREE) return TREE is¶    BASE_STRUCT : TREE;¶    BASE_ID     : TREE;¶    BASE_REGION
: TREE;¶  begin¶¶¬     -- AS A FIRST¬APPROXIMATION, BASE¬STRUCTURE¬IS THE BASE TYPE¶    BASE_STRUCT := GET_BASE_TYPE (TYPE_SPEC);¶¶¬     -- IF IT'S
A POSSIBLE FULL TYPE FOR A PRIVATE TYPE¶    if BASE_STRUCT.TY in CLASS_DERIVABLE_SPEC or else BASE_STRUCT.TY = DN_INCOMPLETE then¶¶¬¬    -- GET THE ID
ENTIFIER ASSOCIATED WITH THE TYPE DECLARATION¶      BASE_ID := D (XD_SOURCE_NAME, BASE_STRUCT);¶¶¬¬    -- IF¬IT'S AN [L_]PRIVATE_TYPE_ID¶¬¬    -- AND
WE'RE NOT ALREADY LOOKING AT¬THE PRIVATE SPEC¶¬¬    -- (NOTE: FULL TYPE SPEC COULD BE A¬DIFFERENT¬PRIVATE)¶      if BASE_ID.TY¬in DN_PRIVATE_TYPE_ID .
. DN_L_PRIVATE_TYPE_ID and¬then D (SM_TYPE_SPEC, BASE_ID) /= BASE_STRUCT then¶¶¬¬¬  -- IF IT WAS NOT DEFINED IN¬AN ENCLOSING PACKAGE¶¬¬¬  -- (NOTE: LX
_SYMREP(BASE_REGION) --> NOT ENCLOSING)¶        BASE_REGION¬:= D (XD_REGION, BASE_ID);¶        if (BASE_REGION.TY /=¬DN_PACKAGE_ID and then (BASE_REGI
ON.TY /= DN_GENERIC_ID or else D (SM_SPEC, BASE_REGION).TY /= DN_PACKAGE_SPEC)) or else D (LX_SYMREP, BASE_REGION).TY /= DN_SYMBOL_REP¶        or els
e DI¬(XD_LEX_LEVEL, GET_DEF_FOR_ID¬(BASE_REGION)) <= 0¶        then¶¶¬¬¬¬¬-- THE STRUCTURE IS¬THE PRIVATE NODE¶¬BASE_STRUCT := D (SM_TYPE_SPEC, BASE_ID
);¶        end if;¶      end¬if;¶    end if;¶¶¬     -- RETURN THE¬BASE STRUCTURE¶    return BASE_STRUCT;¶¶  end GET_BASE_STRUCT;¶      --|||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function GET_ANCESTOR_TYPE (TYPE_SPEC¬: TREE) return TREE
¬is¶    TYPE_STRUCT : TREE := GET_BASE_STRUCT (TYPE_SPEC);¶  begin¶    while¬TYPE_STRUCT.TY in CLASS_DERIVABLE_SPEC and then D¬(SM_DERIVED, TYPE_STRUC
T) /= TREE_VOID loop¶      TYPE_STRUCT := GET_BASE_STRUCT (D¬(SM_DERIVED, TYPE_STRUCT));¶    end loop;¶    return GET_BASE_TYPE (TYPE_STRUCT);¶  end G
ET_ANCESTOR_TYPE;¶¶  function IS_MEMBER_OF_UNSPECIFIED (SPEC_TYPE : TREE; UNSPEC_TYPE : TREE) return Boolean¬is¶    UNSPEC_KIND : NODE_NAME := UNSPEC_
TYPE.TY;¶    SPEC_STRUCT : TREE;¶    SPEC_KIND   : NODE_NAME;¶  begin¶    if UNSPEC_KIND not in CLASS_UNSPECIFIED_TYPE then¶      return False;¶    en
d if;¶¶    SPEC_STRUCT := GET_BASE_STRUCT (SPEC_TYPE);¶    SPEC_KIND   := SPEC_STRUCT.TY;¶    case CLASS_UNSPECIFIED_TYPE'(UNSPEC_KIND) is¶      when
DN_ANY_ACCESS =>¶        return SPEC_KIND = DN_ACCESS or¬SPEC_KIND¬= DN_ANY_ACCESS_OF;¶      when DN_ANY_ACCESS_OF =>¶        if SPEC_KIND = DN_ANY_AC
CESS_OF¬then¶¬return D (XD_ITEM, UNSPEC_TYPE) = D (XD_ITEM, SPEC_TYPE);¶        elsif SPEC_KIND = DN_ACCESS then¶¬return D (XD_ITEM, UNSPEC_TYPE) = GE
T_BASE_TYPE (D (SM_DESIG_TYPE, SPEC_STRUCT));¶        else¶¬¬¬¬¬-- (FALSE¬IF SPEC_TYPE IS ACCESS)¶¬return False;¶        end if;¶      when DN_ANY_COMP
OSITE =>¶        return IS_NONLIMITED_COMPOSITE_TYPE (SPEC_TYPE);¶      when DN_ANY_STRING =>¶        return IS_STRING_TYPE¬(SPEC_TYPE);¶      when DN
_ANY_INTEGER =>¶        return SPEC_KIND = DN_INTEGER or SPEC_KIND = DN_UNIVERSAL_INTEGER;¶      when DN_ANY_REAL =>¶        return SPEC_KIND = DN_FLO
AT or SPEC_KIND = DN_FIXED¬or SPEC_KIND = DN_UNIVERSAL_REAL;¶¶    end case;¶  end IS_MEMBER_OF_UNSPECIFIED;¶¶  function IS_NONLIMITED_COMPOSITE_TYPE¬(
TYPE_SPEC : TREE) return Boolean is¶    TYPE_KIND : NODE_NAME;¶  begin¶    TYPE_KIND := GET_BASE_STRUCT (TYPE_SPEC).TY;¶    if TYPE_KIND = DN_ANY_STRI
NG then¶      return True;¶    elsif¬TYPE_KIND¬= DN_ARRAY or else TYPE_KIND = DN_RECORD then¶      return IS_NONLIMITED_TYPE (TYPE_SPEC);¶    else¶
  return False;¶    end if;¶  end IS_NONLIMITED_COMPOSITE_TYPE;¶¶  function IS_STRING_TYPE (TYPE_SPEC : TREE) return Boolean¬is¶    TYPE_STRUCT : TRE
E := GET_BASE_STRUCT (TYPE_SPEC);¶  begin¶    if TYPE_STRUCT.TY = DN_ANY_STRING then¶      return True;¶    elsif¬TYPE_STRUCT.TY = DN_ARRAY and¬then I
S_EMPTY (TAIL¬(LIST (D (SM_INDEX_S, TYPE_STRUCT)))) then¶      return IS_CHARACTER_TYPE (GET_BASE_TYPE (D (SM_COMP_TYPE, TYPE_STRUCT)));¶    else¶
  return False;¶    end if;¶  end IS_STRING_TYPE;¶¶  function IS_CHARACTER_TYPE (TYPE_SPEC¬: TREE) return Boolean is¶    TYPE_STRUCT : TREE := GET_BAS
E_STRUCT (TYPE_SPEC);¶    ENUM_LIST   : SEQ_TYPE;¶    ENUM_ID     : TREE;¶  begin¶    if TYPE_STRUCT.TY /= DN_ENUMERATION¬then¶      return False;¶
 end if;¶¶¬     -- $$$$ NEED A FASTER TEST FOR TYPE DERIV FROM PREDEF¬CHARACTER¶    ENUM_LIST := LIST (D (SM_LITERAL_S,¬TYPE_SPEC));¶    while¬not IS
_EMPTY (ENUM_LIST) loop¶      POP¬(ENUM_LIST, ENUM_ID);¶      if ENUM_ID.TY¬= DN_CHARACTER_ID then¶        return True;¶      end¬if;¶    end loop;¶¶

   return False;¶  end IS_CHARACTER_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      -
-|¶  procedure REQUIRE_SAME_TYPES (EXP_1 :¬TREE; TYPESET_1 : TYPESET_TYPE; EXP_2 :¬TREE; TYPESET_2 : TYPESET_TYPE; TYPESET_OUT : out¬TYPESET_TYPE) is¶
    TYPESET_1_WORK : TYPESET_TYPE := TYPESET_1;¶    TYPEINTERP_1    : TYPEINTERP_TYPE;¶    TYPE_SPEC_1     : TREE;¶    TYPESET_2_WORK : TYPESET_TYPE;¶
  TYPEINTERP_2    : TYPEINTERP_TYPE;¶    TYPE_SPEC_2     : TREE;¶    NEW_TYPESET     : TYPESET_TYPE := EMPTY_TYPESET;¶¶  begin -- REQUIRE_SAME_TYPES¶
if IS_EMPTY (TYPESET_1) or else IS_EMPTY (TYPESET_2) then¶      TYPESET_OUT := EMPTY_TYPESET;¶      return;¶    end if;¶¶    while¬not IS_EMPTY (TYPES
ET_1_WORK)¬loop¶      POP¬(TYPESET_1_WORK, TYPEINTERP_1);¶      TYPE_SPEC_1¬ := GET_TYPE (TYPEINTERP_1);¶      TYPESET_2_WORK := TYPESET_2;¶      whil
e not IS_EMPTY (TYPESET_2_WORK) loop¶        POP (TYPESET_2_WORK, TYPEINTERP_2);¶        TYPE_SPEC_2¬:= GET_TYPE (TYPEINTERP_2);¶        if TYPE_SPEC_
1 = TYPE_SPEC_2 or¬else IS_MEMBER_OF_UNSPECIFIED¬(TYPE_SPEC_1, TYPE_SPEC_2) then¶¬ADD_EXTRAINFO (TYPEINTERP_1, TYPEINTERP_2);¶¬ADD_TO_TYPESET (NEW_TYP
ESET, TYPEINTERP_1);¶        elsif IS_MEMBER_OF_UNSPECIFIED (TYPE_SPEC_2, TYPE_SPEC_1) then¶¬ADD_EXTRAINFO (TYPEINTERP_2, TYPEINTERP_1);¶¬ADD_TO_TYPES
ET (NEW_TYPESET, TYPEINTERP_2);¶        end if;¶      end¬loop;¶    end loop;¶¶    if IS_EMPTY (NEW_TYPESET)¬then¶      ERROR (D (LX_SRCPOS, EXP_1), "
EXPRESSIONS MUST BE OF THE SAME TYPE");¶    end if;¶    TYPESET_OUT := NEW_TYPESET;¶  end REQUIRE_SAME_TYPES;¶      --||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  procedure REQUIRE_TYPE (TYPE_SPEC : TREE; EXP :¬TREE; TYPESET : in out TYPESE
T_TYPE) is¶    TYPE_STRUCT : TREE;¶    TYPEINTERP  : TYPEINTERP_TYPE;¶    TYPE_NODE   : TREE;¶    TYPE_KIND   : NODE_NAME;¶    NEW_TYPESET : TYPESET_T
YPE := EMPTY_TYPESET;¶  begin¶    if IS_EMPTY (TYPESET) then¶      return;¶    end if;¶¶    while¬not IS_EMPTY (TYPESET) loop¶      POP¬(TYPESET,¬TYPE
INTERP);¶      TYPE_NODE := GET_TYPE (TYPEINTERP);¶      if TYPE_NODE = TYPE_SPEC then¶        ADD_TO_TYPESET (NEW_TYPESET, TYPEINTERP);¶      else¶
      TYPE_KIND := TYPE_NODE.TY;¶        if TYPE_KIND in CLASS_UNSPECIFIED_TYPE then¶¬TYPE_STRUCT := GET_BASE_STRUCT (TYPE_SPEC);¶¬case CLASS_UNSPECIF
IED_TYPE'(TYPE_KIND)¬is¶¬  when DN_ANY_ACCESS =>¶¬    if TYPE_STRUCT.TY = DN_ACCESS then¶¬      ADD_TO_TYPESET (NEW_TYPESET, TYPE_SPEC, GET_EXTRAINFO¬
(TYPEINTERP));¶¬    end if;¶¬  when DN_ANY_COMPOSITE =>¶¬    if IS_NONLIMITED_COMPOSITE_TYPE (TYPE_SPEC) then¶¬      ADD_TO_TYPESET (NEW_TYPESET, TYPE
_SPEC, GET_EXTRAINFO¬(TYPEINTERP));¶¬    end if;¶¬  when DN_ANY_STRING =>¶¬    if IS_STRING_TYPE (TYPE_SPEC) then¶¬      ADD_TO_TYPESET (NEW_TYPESET,
TYPE_SPEC, GET_EXTRAINFO¬(TYPEINTERP));¶¬    end if;¶¬  when DN_ANY_ACCESS_OF =>¶¬    if TYPE_STRUCT.TY = DN_ACCESS and then GET_BASE_TYPE (D¬(SM_DESI
G_TYPE, TYPE_STRUCT))¬= GET_BASE_TYPE (D (XD_ITEM, TYPE_NODE)) then¶¬      ADD_TO_TYPESET (NEW_TYPESET, TYPE_SPEC, GET_EXTRAINFO¬(TYPEINTERP));¶¬    e
nd if;¶¬  when DN_ANY_INTEGER =>¶¬    if IS_INTEGER_TYPE (TYPE_SPEC) then¶¬      ADD_TO_TYPESET (NEW_TYPESET, TYPE_SPEC, GET_EXTRAINFO¬(TYPEINTERP));¶
¬    end if;¶¬  when DN_ANY_REAL =>¶¬    if IS_REAL_TYPE¬(TYPE_SPEC) then¶¬      ADD_TO_TYPESET (NEW_TYPESET, TYPE_SPEC, GET_EXTRAINFO¬(TYPEINTERP));¶
¬    end if;¶¬end case;¶        end if;¶      end¬if;¶    end loop;¶¶    TYPESET := NEW_TYPESET;¶    if IS_EMPTY (TYPESET) then¶      ERROR (D (LX_SRC
POS, EXP), "EXP NOT OF REQUIRED TYPE");¶    end if;¶  end REQUIRE_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||¶      --|¶  function IS_NONLIMITED_TYPE¬(ITEM : TREE) return Boolean is¶¶    TYPE_SPEC : constant TREE¬:= GET_BASE_STRUCT (ITEM
);¶¶    function GET_VARIABLE_TYPE_SPEC (VARIABLE_DECL : TREE) return TREE is¶      SOURCE_NAME_LIST : SEQ_TYPE := LIST (D (AS_SOURCE_NAME_S, VARIABLE
_DECL));¶    begin¶      return GET_BASE_TYPE (D¬(SM_OBJ_TYPE, HEAD (SOURCE_NAME_LIST)));¶    end GET_VARIABLE_TYPE_SPEC;¶¶    function IS_NONLIMITED_
COMP_LIST (COMP_LIST :¬TREE) return Boolean is¶      ITEM_LIST    : SEQ_TYPE¬:= LIST (COMP_LIST);¶      ITEM         : TREE;¶      DECL_LIST    : SEQ_
TYPE;¶      DECL         : TREE;¶      VARIANT_PART : TREE;¶      VARIANT_LIST : SEQ_TYPE;¶      VARIANT      : TREE;¶    begin¶      while not IS_EMP
TY (ITEM_LIST) loop¶        POP (ITEM_LIST, ITEM);¶        DECL_LIST := LIST (D (AS_DECL_S, ITEM));¶        while not IS_EMPTY (DECL_LIST) loop¶¬POP (
DECL_LIST, DECL);¶¬if DECL.TY = DN_VARIABLE_DECL¬then¶¬  if not IS_NONLIMITED_TYPE (GET_VARIABLE_TYPE_SPEC (DECL))¬then¶¬    return False;¶¬  end if;¶
¬end if;¶        end loop;¶        VARIANT_PART := D (AS_VARIANT_PART, ITEM);¶        if VARIANT_PART.TY = DN_VARIANT_PART then¶¬VARIANT_LIST := LIST
(D (AS_VARIANT_S, VARIANT_PART));¶¬while not¬IS_EMPTY (VARIANT_LIST) loop¶¬  POP (VARIANT_LIST, VARIANT);¶¬  if VARIANT.TY = DN_VARIANT then¶¬    if n
ot IS_NONLIMITED_COMP_LIST (D (AS_COMP_LIST, VARIANT)) then¶¬      return False;¶¶¬    end if;¶¬  end if;¶¬end loop;¶        end if;¶      end¬loop;¶
    return True;¶    end IS_NONLIMITED_COMP_LIST;¶¶  begin -- IS_NONLIMITED_TYPE¶¶    if TYPE_SPEC = TREE_VOID then¶      return True;¶    end if;¶¶
   case CLASS_TYPE_SPEC'(TYPE_SPEC.TY)¬is¶      when DN_TASK_SPEC | DN_L_PRIVATE | DN_INCOMPLETE =>¶        return False;¶      when DN_RECORD =>¶
    if not DB (SM_IS_LIMITED, TYPE_SPEC) then¶¬return True;¶        else¶¬return IS_NONLIMITED_COMP_LIST (D (SM_COMP_LIST, TYPE_SPEC));¶        end if
;¶      when DN_ARRAY¬=>¶        if DB (SM_IS_LIMITED,¬TYPE_SPEC) then¶¬return False;¶        else¶¬return IS_NONLIMITED_TYPE (D (SM_COMP_TYPE, TYPE_S
PEC));¶        end if;¶      when others =>¶        return True;¶    end case;¶  end IS_NONLIMITED_TYPE;¶      --||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_LIMITED_TYPE (ITEM : TREE) return Boolean is¶  begin¶    return not IS
_NONLIMITED_TYPE (ITEM);¶  end IS_LIMITED_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶
    --|¶  function IS_PRIVATE_TYPE (ITEM : TREE) return Boolean is¶¬      -- RETURNS TRUE IF ITEM¬IS PRIVATE¶¬      -- $$$$ WORRY¬ABOUT WHAT THIS MEA
NS¶    TYPE_SPEC : constant TREE¬:= GET_BASE_STRUCT (ITEM);¶¶  begin -- IS_PRIVATE_TYPE¶¶    case CLASS_TYPE_SPEC'(TYPE_SPEC.TY)¬is¶      when DN_TASK
_SPEC | DN_L_PRIVATE | DN_INCOMPLETE | DN_PRIVATE =>¶        return True;¶      when DN_ARRAY¬=>¶        case CLASS_TYPE_SPEC (GET_BASE_STRUCT (D (SM_
COMP_TYPE, TYPE_SPEC)).TY)¬is¶¬when DN_TASK_SPEC |¬DN_L_PRIVATE | DN_INCOMPLETE | DN_PRIVATE =>¶¬  return True;¶¬when others =>¶¬  return False;¶
   end case;¶      when others =>¶        return False;¶    end case;¶  end IS_PRIVATE_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_INTEGER_TYPE (ITEM : TREE) return Boolean is¶    TYPE_SPEC : constant TREE¬     :=
 GET_BASE_STRUCT (ITEM);¶    TYPE_KIND : constant NODE_NAME := TYPE_SPEC.TY;¶  begin¶    return TYPE_KIND = DN_INTEGER or TYPE_KIND = DN_UNIVERSAL_INT
EGER¬or TYPE_KIND = DN_ANY_INTEGER;¶  end IS_INTEGER_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||

||||||||¶      --|¶  function IS_REAL_TYPE (ITEM¬: TREE) return Boolean is¶    TYPE_SPEC : constant TREE¬     := GET_BASE_STRUCT (ITEM);¶    TYPE_KIN
D : constant NODE_NAME := TYPE_SPEC.TY;¶  begin¶    return TYPE_KIND = DN_FLOAT or TYPE_KIND = DN_FIXED or TYPE_KIND = DN_UNIVERSAL_REAL or TYPE_KIND
= DN_ANY_REAL;¶  end IS_REAL_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  fu
nction IS_SCALAR_TYPE (ITEM : TREE)¬return Boolean is¶    TYPE_SPEC : constant TREE¬     := GET_BASE_STRUCT (ITEM);¶    TYPE_KIND : constant NODE_NAME
 := TYPE_SPEC.TY;¶  begin¶    return TYPE_KIND in CLASS_SCALAR or¬TYPE_KIND¬= DN_UNIVERSAL_INTEGER or TYPE_KIND = DN_UNIVERSAL_REAL or TYPE_KIND in DN
_ANY_INTEGER ..¬DN_ANY_REAL;¶  end IS_SCALAR_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|¶      --|¶  function IS_BOOLEAN_TYPE (ITEM : TREE) return Boolean is¶  begin¶¬      -- TYPE IS BOOLEAN IF IT IS DERIVED FROM PREDEFINED BOOLEAN¶
return GET_ANCESTOR_TYPE (ITEM) = PREDEFINED_BOOLEAN;¶  end IS_BOOLEAN_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||¶      --|¶  function IS_UNIVERSAL_TYPE (ITEM : TREE) return¬Boolean is¶    ITEM_KIND : NODE_NAME := ITEM.TY;¶  begin¶    r
eturn ITEM_KIND = DN_UNIVERSAL_INTEGER or ITEM_KIND = DN_UNIVERSAL_REAL or¬ITEM_KIND¬= DN_ANY_INTEGER or¬ITEM_KIND¬= DN_ANY_REAL;¶  end IS_UNIVERSAL_T
YPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_NON_UNIVERSAL_TYPE (
ITEM : TREE) return Boolean is¶  begin¶    return not IS_UNIVERSAL_TYPE (ITEM);¶  end IS_NON_UNIVERSAL_TYPE;¶      --||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_DISCRETE_TYPE (ITEM : TREE) return Boolean is¶    BASE_STRUCT : TR
EE := GET_BASE_STRUCT (ITEM);¶  begin¶    case BASE_STRUCT.TY is¶      when DN_ENUMERATION | DN_INTEGER | DN_UNIVERSAL_INTEGER | DN_ANY_INTEGER =>¶
     return True;¶      when others =>¶        return False;¶    end case;¶  end IS_DISCRETE_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_TASK_TYPE (ITEM¬: TREE) return Boolean is¶    BASE_STRUCT : TREE := GET_BASE
_STRUCT (ITEM);¶  begin¶    return BASE_STRUCT.TY = DN_TASK_SPEC;¶  end IS_TASK_TYPE;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
||||||||||||||||||||||||||||||||||||||||¶      --|¶  procedure REQUIRE_ID (ID_KIND : NODE_NAME; EXP : TREE; DEFSET : in out DEFSET_TYPE) is¶    DEFINTERP
 : DEFINTERP_TYPE;¶    NEW_DEFSET : DEFSET_TYPE := EMPTY_DEFSET;¶  begin¶    if IS_EMPTY (DEFSET) then¶      return;¶    end if;¶¶    while¬not IS_EMP
TY (DEFSET) loop¶      POP¬(DEFSET, DEFINTERP);¶      if D (XD_SOURCE_NAME, GET_DEF (DEFINTERP)).TY = ID_KIND then¶        ADD_TO_DEFSET (NEW_DEFSET,
DEFINTERP);¶      end¬if;¶    end loop;¶¶    DEFSET := NEW_DEFSET;¶    if IS_EMPTY (DEFSET) then¶      ERROR (D (LX_SRCPOS, EXP), "NAME IS NOT " &¬NOD
E_NAME'IMAGE (ID_KIND));¶    end if;¶  end REQUIRE_ID;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
```

```
|||||¶      --|¶  function IS_TYPE_DEF (ITEM : TREE) return Boolean is¶    ITEM_KIND : NODE_NAME := D (XD_SOURCE_NAME, ITEM).TY;¶  begin¶    return IT
EM_KIND in CLASS_TYPE_NAME;¶  end IS_TYPE_DEF;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶
    --|¶  function IS_ENTRY_DEF (ITEM¬: TREE) return Boolean is¶  begin¶    return D (XD_SOURCE_NAME,¬ITEM).TY = DN_ENTRY_ID;¶  end IS_ENTRY_DEF;¶
  --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_PROC_OR_ENTRY_DEF (ITEM :¬TRE
E) return Boolean is¶    SOURCE_NAME_KIND : NODE_NAME := D (XD_SOURCE_NAME, ITEM).TY;¶  begin¶    if SOURCE_NAME_KIND = DN_PROCEDURE_ID or else¬SOURCE
_NAME_KIND = DN_ENTRY_ID then¶      return True;¶    elsif¬SOURCE_NAME_KIND = DN_GENERIC_ID and then D (XD_HEADER, ITEM).TY = DN_PROCEDURE_SPEC then¶
     return True;¶    else¶      return False;¶    end if;¶  end IS_PROC_OR_ENTRY_DEF;¶      --|||||||||||||||||||||||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||¶      --|¶  function IS_FUNCTION_OR_ARRAY_DEF (ITEM : TREE)¬return Boolean is¶    ITEM_KIND   : NODE_NAME := D (
XD_SOURCE_NAME,¬ITEM).TY;¶    ITEM_STRUCT : TREE;¶  begin¶    if ITEM_KIND = DN_FUNCTION_ID or ITEM_KIND = DN_OPERATOR_ID or ITEM_KIND = DN_BLTN_OPERA
TOR_ID then¶      return True;¶    elsif¬ITEM_KIND¬= DN_GENERIC_ID and¬then D (XD_HEADER, ITEM).TY =¬DN_FUNCTION_SPEC then¶      return True;¶    elsi
f¬ITEM_KIND¬in CLASS_OBJECT_NAME then¶      ITEM_STRUCT := GET_BASE_STRUCT (D¬(XD_SOURCE_NAME, ITEM));¶      if ITEM_STRUCT.TY = DN_ACCESS then¶
  ITEM_STRUCT¬:= GET_BASE_STRUCT (D (SM_DESIG_TYPE, ITEM_STRUCT));¶      end¬if;¶      return ITEM_STRUCT.TY =¬DN_ARRAY;¶    else¶      return False;¶
    end if;¶  end IS_FUNCTION_OR_ARRAY_DEF;¶      --||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||||¶
 --|¶  function IS_FUNCTION_OR_ENUMERATION_DEF (ITEM :¬TREE) return Boolean is¶    ITEM_KIND : NODE_NAME := D (XD_SOURCE_NAME, ITEM).TY;¶  begin¶    i
f ITEM_KIND = DN_FUNCTION_ID or ITEM_KIND = DN_OPERATOR_ID or ITEM_KIND = DN_BLTN_OPERATOR_ID or ITEM_KIND = DN_ENUMERATION_ID then¶      return True;
¶    else¶      return False;¶    end if;¶  end IS_FUNCTION_OR_ENUMERATION_DEF;¶¶  procedure REQUIRE_NONLIMITED_TYPE (EXP : TREE; TYPESET : in out TYP
ESET_TYPE)¬is¶    procedure N_REQUIRE_NONLIMITED_TYPE¬is new REQ_TYPE_XXX¬(IS_NONLIMITED_TYPE, "NONLIMITED TYPE REQUIRED");¶  begin¶    N_REQUIRE_NONL
IMITED_TYPE¬(EXP, TYPESET);¶  end REQUIRE_NONLIMITED_TYPE;¶  procedure REQUIRE_INTEGER_TYPE (EXP :¬TREE; TYPESET : in out TYPESET_TYPE) is¶    procedu
re N_REQUIRE_INTEGER_TYPE is¬new REQ_TYPE_XXX (IS_INTEGER_TYPE, "INTEGER TYPE REQUIRED");¶  begin¶    N_REQUIRE_INTEGER_TYPE (EXP, TYPESET);¶  end REQ
UIRE_INTEGER_TYPE;¶  procedure REQUIRE_BOOLEAN_TYPE (EXP :¬TREE; TYPESET : in out TYPESET_TYPE) is¶    procedure N_REQUIRE_BOOLEAN_TYPE is¬new REQ_TYP
E_XXX (IS_BOOLEAN_TYPE, "BOOLEAN TYPE REQUIRED");¶  begin¶    N_REQUIRE_BOOLEAN_TYPE (EXP, TYPESET);¶  end REQUIRE_BOOLEAN_TYPE;¶  procedure REQUIRE_R
EAL_TYPE¬(EXP : TREE; TYPESET : in out¬TYPESET_TYPE) is¶    procedure N_REQUIRE_REAL_TYPE is new REQ_TYPE_XXX (IS_REAL_TYPE, "REAL TYPE REQUIRED");¶
begin¶    N_REQUIRE_REAL_TYPE (EXP,¬TYPESET);¶  end REQUIRE_REAL_TYPE;¶  procedure REQUIRE_SCALAR_TYPE (EXP : TREE; TYPESET : in out TYPESET_TYPE) is¶
    procedure N_REQUIRE_SCALAR_TYPE is new REQ_TYPE_XXX (IS_SCALAR_TYPE, "SCALAR TYPE REQUIRED");¶  begin¶    N_REQUIRE_SCALAR_TYPE (EXP, TYPESET);¶
end REQUIRE_SCALAR_TYPE;¶  procedure REQUIRE_UNIVERSAL_TYPE (EXP¬: TREE; TYPESET : in out TYPESET_TYPE) is¶    procedure N_REQUIRE_UNIVERSAL_TYPE is n
ew REQ_TYPE_XXX (IS_UNIVERSAL_TYPE, "UNIVERSAL TYPE REQUIRED");¶  begin¶    N_REQUIRE_UNIVERSAL_TYPE (EXP, TYPESET);¶  end REQUIRE_UNIVERSAL_TYPE;¶  p
rocedure REQUIRE_NON_UNIVERSAL_TYPE (EXP : TREE; TYPESET¬: in out TYPESET_TYPE) is¶    procedure N_REQUIRE_NON_UNIVERSAL_TYPE is new¬REQ_TYPE_XXX (IS_
NON_UNIVERSAL_TYPE, "NON-UNIVERSAL TYPE REQUIRED");¶  begin¶    N_REQUIRE_NON_UNIVERSAL_TYPE (EXP, TYPESET);¶  end REQUIRE_NON_UNIVERSAL_TYPE;¶  proce

dure REQUIRE_DISCRETE_TYPE (EXP : TREE; TYPESET : in¬out TYPESET_TYPE) is¶    procedure N_REQUIRE_DISCRETE_TYPE is new REQ_TYPE_XXX (IS_DISCRETE_TYPE,
 "DISCRETE TYPE REQUIRED");¶  begin¶    N_REQUIRE_DISCRETE_TYPE (EXP, TYPESET);¶  end REQUIRE_DISCRETE_TYPE;¶  procedure REQUIRE_TASK_TYPE¬(EXP : TREE
; TYPESET : in out¬TYPESET_TYPE) is¶    procedure N_REQUIRE_TASK_TYPE is new REQ_TYPE_XXX (IS_TASK_TYPE, "TASK TYPE REQUIRED");¶  begin¶    N_REQUIRE_
TASK_TYPE (EXP,¬TYPESET);¶  end REQUIRE_TASK_TYPE;¶  procedure REQUIRE_TYPE_DEF (EXP : TREE; DEFSET : in out DEFSET_TYPE) is¶    procedure N_REQUIRE_T
YPE_DEF is new¬REQ_DEF_XXX (IS_TYPE_DEF, "TYPE OR SUBTYPE NAME REQUIRED");¶  begin¶    N_REQUIRE_TYPE_DEF (EXP, DEFSET);¶  end REQUIRE_TYPE_DEF;¶  pro
cedure REQUIRE_ENTRY_DEF¬(EXP : TREE; DEFSET¬: in out DEFSET_TYPE) is¶    procedure N_REQUIRE_ENTRY_DEF is new REQ_DEF_XXX (IS_ENTRY_DEF, "ENTRY NAME
REQUIRED");¶  begin¶    N_REQUIRE_ENTRY_DEF (EXP,¬DEFSET);¶  end REQUIRE_ENTRY_DEF;¶  procedure REQUIRE_PROC_OR_ENTRY_DEF (EXP : TREE; DEFSET :¬in out
 DEFSET_TYPE)¬is¶    procedure N_REQUIRE_PROC_OR_ENTRY_DEF is new REQ_DEF_XXX (IS_PROC_OR_ENTRY_DEF, "PROCEDURE OR ENTRY NAME REQUIRED");¶  begin¶
N_REQUIRE_PROC_OR_ENTRY_DEF (EXP, DEFSET);¶  end REQUIRE_PROC_OR_ENTRY_DEF;¶  procedure REQUIRE_FUNCTION_OR_ARRAY_DEF (EXP : TREE; DEFSET : in out DEF
SET_TYPE) is¶    procedure N_REQUIRE_FUNCTION_OR_ARRAY_DEF is new REQ_DEF_XXX (IS_FUNCTION_OR_ARRAY_DEF, "FUNCTION OR ARRAY OR ACCESS ARRAY REQUIRED")
;¶  begin¶    N_REQUIRE_FUNCTION_OR_ARRAY_DEF (EXP, DEFSET);¶  end REQUIRE_FUNCTION_OR_ARRAY_DEF;¶  procedure REQUIRE_FUNCTION_OR_ENUMERATION_DEF (EXP
 : TREE; DEFSET :¬in out DEFSET_TYPE)¬is¶    procedure N_REQUIRE_FUNCTION_OR_ENUMERATION_DEF is new REQ_DEF_XXX (IS_FUNCTION_OR_ENUMERATION_DEF, "FUNC
TION OR ENUMERATION LITERAL REQUIRED");¶  begin¶    N_REQUIRE_FUNCTION_OR_ENUMERATION_DEF (EXP, DEFSET);¶  end REQUIRE_FUNCTION_OR_ENUMERATION_DEF;¶¶
    --|----------------------------------------------------------------------------------------------------¶end REQ_UTIL;¶
```